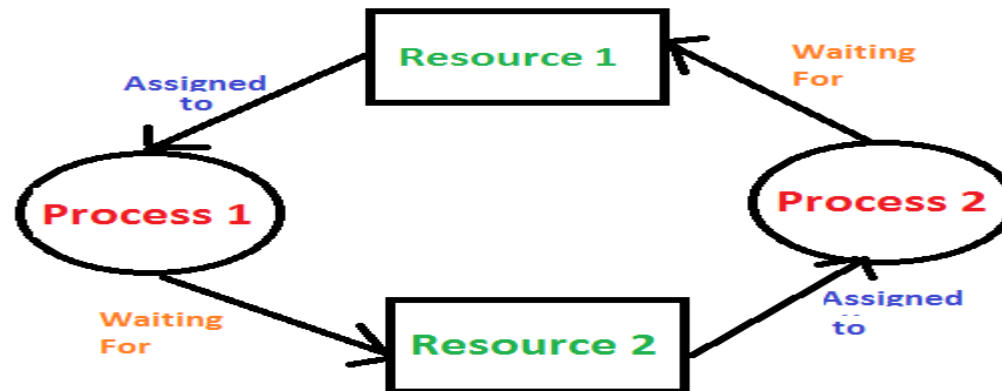


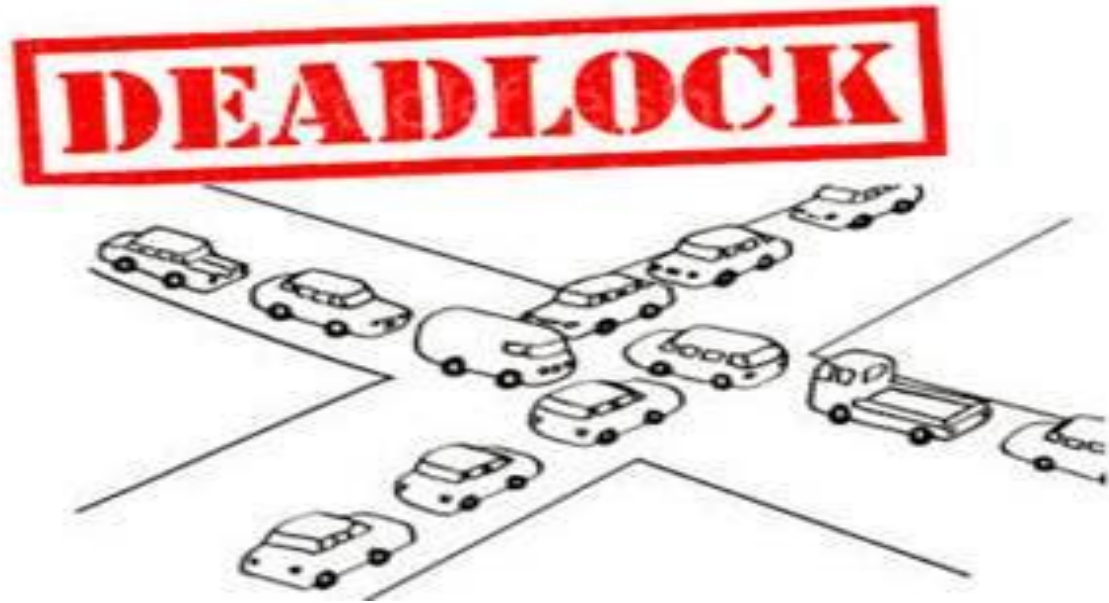
Deadlock

Introduction of Deadlock in Operating System

- A process in operating system uses resources in the following way.
 - 1) Requests a resource
 - 2) Use the resource
 - 3) Releases the resource
- **Deadlock** is a situation where a set of processes are blocked because each process is holding a resource and waiting for another process.



Example:



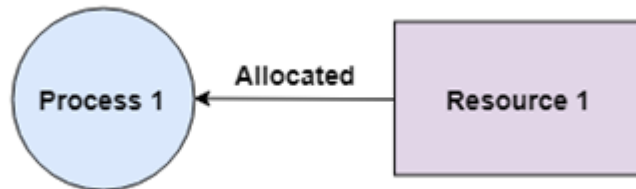
Difference between Starvation and Deadlock

Sr.	Deadlock	Starvation
1	Deadlock is a situation where no process got blocked and no process proceeds	Starvation is a situation where the low priority process got blocked and the high priority processes proceed.
2	Deadlock is an infinite waiting.	Starvation is a long waiting but not infinite.
3	Every Deadlock is always a starvation.	Every starvation need not be deadlock.
4	The requested resource is blocked by the other process.	The requested resource is continuously be used by the higher priority processes.
5	Deadlock happens when Mutual exclusion, hold and wait, No preemption and circular wait occurs simultaneously.	It occurs due to the uncontrolled priority and resource management.

Necessary conditions for Deadlocks

1. Mutual Exclusion:

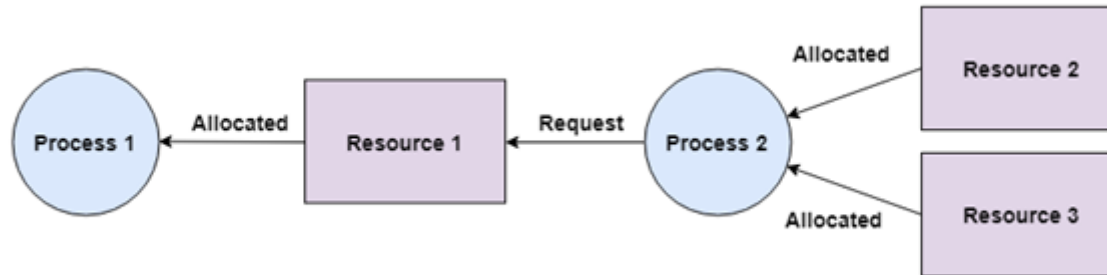
- A resource can only be shared in mutually exclusive manner. It implies, if two process cannot use the same resource at the same time.
- In the diagram below, there is a single instance of Resource 1 and it is held by Process 1 only.



Necessary conditions for Deadlocks

2. Hold and Wait: A process waits for some resources while holding another resource at the same time.

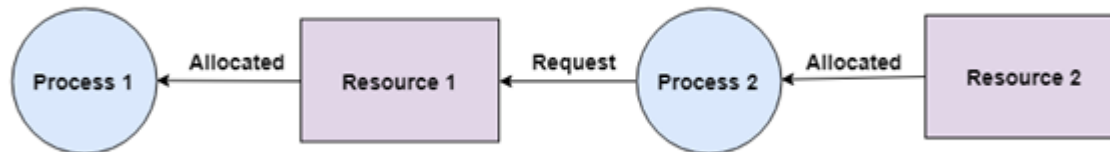
➤ In the diagram given below, Process 2 holds Resource 2 and Resource 3 and is requesting the Resource 1 which is held by Process 1.



Necessary conditions for Deadlocks

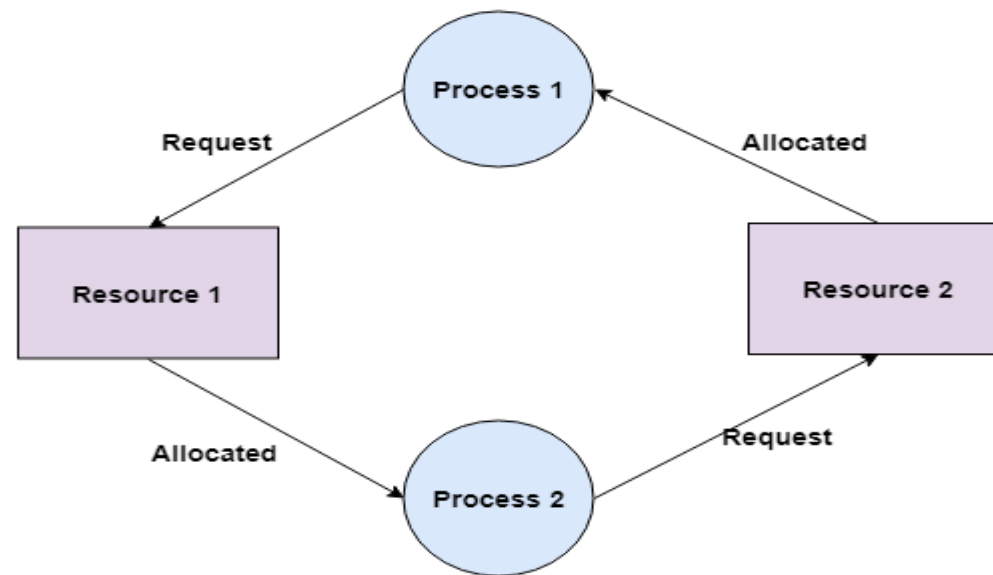
3. No preemption: The process which once scheduled will be executed till the completion. No other process can be scheduled by the scheduler meanwhile.

➤ In the diagram below, Process 2 cannot preempt Resource 1 from Process 1. It will only be released when Process 1 relinquishes it voluntarily after its execution is complete.



Necessary conditions for Deadlocks:

4. Circular Wait: All the processes must be waiting for the resources in a cyclic manner so that the last process is waiting for the resource which is being held by the first process.



Methods for handling deadlock

➤ There are three ways to handle deadlock

- 1) **Deadlock prevention or avoidance:** The idea is to not let the system into a deadlock state.
 - Prevention is done by negating one of above-mentioned necessary conditions for deadlock.
 - Avoidance is kind of futuristic in nature. By using strategy of “Avoidance”, we have to make an assumption.
 - We need to ensure that all information about resources which process will need are known to us prior to execution of the process.
 - We use **Banker's algorithm** (Which is in-turn a gift from Dijkstra) in order to avoid deadlock.
- 2) **Deadlock detection and recovery:** Let deadlock occur, then do preemption to handle it once occurred.
- 3) **Ignore the problem altogether:** If deadlock is very rare, then let it happen and reboot the system. This is the approach that both Windows and UNIX take.

Deadlock prevention or avoidance:

- **Deadlock Prevention:**

We can prevent Deadlock by eliminating any of the above four conditions.

1. Eliminate Mutual Exclusion

- It is not possible to dis-satisfy the mutual exclusion because some resources, such as the tape drive and printer, are inherently non-shareable.

2. Eliminate Hold and wait

- a) Allocate all required resources to the process before the start of its execution, this way hold and wait condition is eliminated but it will lead to low device utilization.
- **for example**, if a process requires printer at a later time and we have allocated printer before the start of its execution printer will remain blocked till it has completed its execution.
- b) The process will make a new request for resources after releasing the current set of resources. This solution may lead to **starvation**.

Deadlock prevention or avoidance:

3. Eliminate No Preemption

- Preempt resources from the process when resources required by other high priority processes.

4. Eliminate Circular Wait

- Each resource will be assigned with a numerical number. A process can request the resources increasing/decreasing. order of numbering.
- For Example, if P1 process is allocated R5 resources, now next time if P1 ask for R4, R3 lesser than R5 such request will not be granted, only request for resources more than R5 will be granted.

Deadlock avoidance

- In deadlock avoidance, the request for any resource will be granted if the resulting state of the system doesn't cause deadlock in the system.
- The state of the system will continuously be checked for safe and unsafe states.
- **Banker's Algorithm** is used majorly in the banking system to avoid deadlock. It helps you to identify whether a loan will be given or not.
- This algorithm is used to test for safely simulating the allocation for determining the maximum amount available for all resources.
- It also checks for all the possible activities before determining whether allocation should be continued or not.

Banker's Algorithm

- Following **Data structures** are used to implement the Banker's Algorithm:
- Let '**n**' be the number of processes in the system and '**m**' be the number of resources types.
- **Available :**
 - It is a 1-d array of size '**m**' indicating the number of available resources of each type.
 - $\text{Available}[j] = k$ means there are '**k**' instances of resource type **R_j**
- **Max :**
 - It is a 2-d array of size '**n*m**' that defines the maximum demand of each process in a system.
 - $\text{Max}[i, j] = k$ means process **P_i** may request at most '**k**' instances of resource type **R_j**.
- **Allocation :**
 - It is a 2-d array of size '**n*m**' that defines the number of resources of each type currently allocated to each process.
 - $\text{Allocation}[i, j] = k$ means process **P_i** is currently allocated '**k**' instances of resource type **R_j**

Banker's Algorithm

➤ Need :

- It is a 2-d array of size ' $n*m$ ' that indicates the remaining resource need of each process.
 - $\text{Need}[i, j] = k$ means process P_i currently need ' k ' instances of resource type R_j
 - $\text{Need}[i, j] = \text{Max}[i, j] - \text{Allocation}[i, j]$
- Allocation_i specifies the resources currently allocated to process P_i and Need_i specifies the additional resources that process P_i may still request to complete its task.
- Banker's algorithm consists of **Safety algorithm** and **Resource request algorithm**

Safety Algorithm

- The algorithm for finding out whether or not a system is in a safe state can be described as follows:

1) Let Work and Finish be vectors of length 'm' and 'n' respectively.

Initialize: Work = Available

Finish[i] = false; for i=1, 2, 3, 4....n

2) Find an i such that both

a) Finish[i] = false

b) Need_i ≤ Work

if no such i exists goto step (4)

3) Work = Work + Allocation[i]

Finish[i] = true

goto step (2)

4) if Finish [i] = true for all i

then the system is in a safe state

Resource-Request Algorithm

- Let $Request_i$ be the request array for process P_i . $Request_i[j] = k$ means process P_i wants k instances of resource type R_j .
- When a request for resources is made by process P_i , the following actions are taken:

1) If $Request_i \leq Need_i$

Goto step (2) ; otherwise, raise an error condition, since the process has exceeded its maximum claim.

2) If $Request_i \leq Available$

Goto step (3); otherwise, P_i must wait, since the resources are not available.

3) Have the system pretend to have allocated the requested resources to process P_i by modifying the state as follows:

$Available = Available - Request_i$

$Allocation_i = Allocation_i + Request_i$

$Need_i = Need_i - Request_i$

Example:

- Considering a system with five processes P_0 through P_4 and three resources of type A, B, C. Resource type A has 10 instances, B has 5 instances and type C has 7 instances. Suppose at time t_0 following snapshot of the system has been taken:

Process	Allocation	Max	Available
	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Answer the following questions using the banker's algorithm:

- 1.What will be the content of the Need matrix?**
- 2. Is the system in a safe state? If Yes, then what is the safe sequence?**
- 3.What will happen if process P1 requests one additional instance of resource type A and two instances of resource type C?**

Solution of Question 1:

- $\text{Need } [i, j] = \text{Max } [i, j] - \text{Allocation } [i, j]$
So, the content of Need Matrix is:

Process	Need		
	A	B	C
P ₀	7	4	3
P ₁	1	2	2
P ₂	6	0	0
P ₃	0	1	1
P ₄	4	3	1

Solution of Question 2:

$m=3, n=5$ Step 1 of Safety Algo

Work = Available

Work =

3	3	2
---	---	---

0 1 2 3 4

Finish =

false	false	false	false	false
-------	-------	-------	-------	-------

For $i=0$ Step 2

Need₀ = 7, 4, 3 ✗

Finish [0] is false and Need₀ > Work

So P₀ must wait But Need ≤ Work

For $i=1$ Step 2

Need₁ = 1, 2, 2 ✓

Finish [1] is false and Need₁ < Work

So P₁ must be kept in safe sequence

Step 3

Work = Work + Allocation₁

Work =

A	B	C
5	3	2

0 1 2 3 4

Finish =

false	true	false	false	false
-------	------	-------	-------	-------

For $i=2$ Step 2

Need₂ = 6, 0, 0 ✗

Finish [2] is false and Need₂ > Work

So P₂ must wait

For $i=3$ Step 2

Need₃ = 0, 1, 1 ✓

Finish [3] is false and Need₃ < Work

So P₃ must be kept in safe sequence

Step 3

Work = Work + Allocation₃

Work =

A	B	C
7	4	3

0 1 2 3 4

Finish =

false	true	false	true	false
-------	------	-------	------	-------

For $i=4$ Step 2

Need₄ = 4, 3, 1 ✓

Finish [4] is false and Need₄ < Work

So P₄ must be kept in safe sequence

Step 3

Work = Work + Allocation₄

Work =

A	B	C
7	4	5

0 1 2 3 4

Finish =

false	true	false	true	true
-------	------	-------	------	------

For $i=0$ Step 2

Need₀ = 7, 4, 3 ✓

Finish [0] is false and Need < Work

So P₀ must be kept in safe sequence

Step 3

Work = Work + Allocation₀

Work =

A	B	C
7	5	5

0 1 2 3 4

Finish =

true	true	false	true	true
------	------	-------	------	------

For $i=2$ Step 2

Need₂ = 6, 0, 0 ✓

Finish [2] is false and Need₂ < Work

So P₂ must be kept in safe sequence

Step 3

Work = Work + Allocation₂

Work =

A	B	C
10	5	7

0 1 2 3 4

Finish =

true	true	true	true	true
------	------	------	------	------

Step 4

Finish [i] = true for $0 \leq i \leq n$

Hence the system is in Safe state

The safe sequence is P₁, P₃, P₄, P₀, P₂

Question 3: What will happen if process P1 requests one additional instance of resource type A and two instances of resource type C?

Solution:

$\text{Request}_1 = \begin{matrix} & A & B & C \\ 1, & 0, & 2 \end{matrix}$

To decide whether the request is granted we use Resource Request algorithm

$\begin{matrix} & A & B & C \\ 1, & 0, & 2 & & 1, & 2, & 2 \end{matrix}$
 $\text{Request}_1 < \text{Need}_1$ ✓ Step 1

$\begin{matrix} & A & B & C \\ 1, & 0, & 2 & & 3, & 3, & 2 \end{matrix}$
 $\text{Request}_1 < \text{Available}$ ✓ Step 2

Step 3

Available = Available – Request₁
Allocation₁ = Allocation₁ + Request₁
Need₁ = Need₁ - Request₁

Process	Allocation	Need	Available
	A B C	A B C	A B C
P ₀	0 1 0	7 4 3	2 3 0
P ₁	3 0 2	0 2 0	
P ₂	3 0 2	6 0 0	
P ₃	2 1 1	0 1 1	
P ₄	0 0 2	4 3 1	

Process	Allocation	Max	Available	Process
	A B C	A B C	A B C	
P ₀	0 1 0	7 5 3	3 3 2	P ₀
P ₁	2 0 0	3 2 2		P ₁
P ₂	3 0 2	9 0 2		P ₂
P ₃	2 1 1	2 2 2		P ₃
P ₄	0 0 2	4 3 3		P ₄

We must determine whether this new system state is safe. To do so, we again execute Safety algorithm on the above data structures.

Solution of Question 3:

Step 1 of Safety Algo

$m=3, n=5$
 Work = Available

Work =

2	3	0
---	---	---

Finish =

false	false	false	false	false
-------	-------	-------	-------	-------

Step 2:

For $i=0$
 Need₀ = 7, 4, 3
 Finish [0] is false and **Need₀ > Work**
 So P₀ must wait

Step 2:

For $i=1$
 Need₁ = 0, 2, 0
 Finish [1] is false and **Need₁ < Work**
 So P₁ must be kept in safe sequence

Step 3

Work = Work + Allocation₁

Work =

5	3	2
---	---	---

Finish =

false	true	false	false	false
-------	------	-------	-------	-------

Step 2:

For $i=2$
 Need₂ = 6, 0, 0
 Finish [2] is false and **Need₂ > Work**
 So P₂ must wait

Step 2:

For $i=3$
 Need₃ = 0, 1, 1
 Finish [3] = false and **Need₃ < Work**
 So P₃ must be kept in safe sequence

Step 3

Work = Work + Allocation₃

Work =

7	4	3
---	---	---

Finish =

false	true	false	true	false
-------	------	-------	------	-------

Step 2:

For $i=4$
 Need₄ = 4, 3, 1
 Finish [4] = false and **Need₄ < Work**
 So P₄ must be kept in safe sequence

Step 3

Work = Work + Allocation₄

Work =

7	4	5
---	---	---

Finish =

false	true	false	true	true
-------	------	-------	------	------

Step 2:

For $i=0$
 Need₀ = 7, 4, 3
 Finish [0] is false and **Need₀ < Work**
 So P₀ must be kept in safe sequence

Step 3

Work = Work + Allocation₀

Work =

7	5	5
---	---	---

Finish =

true	true	false	true	true
------	------	-------	------	------

Step 2:

For $i=2$
 Need₂ = 6, 0, 0
 Finish [2] is false and **Need₂ < Work**
 So P₂ must be kept in safe sequence

Step 3

Work = Work + Allocation₂

Work =

10	5	7
----	---	---

Finish =

true	true	true	true	true
------	------	------	------	------

Step 4

Finish [i] = true for $0 \leq i \leq n$
 Hence the system is in Safe state

The safe sequence is P₁, P₃, P₄, P₀, P₂

Solution of Question 3:

- Hence the new system state is safe, so we can immediately grant the request for process P_1 .

Example 2:

1. What will be the content of the Need matrix?

2. Is the system in a safe state? If Yes, then what is the safe sequence?

Processes	Allocation	Max	Available
	A B C	A B C	A B C
P ₀	1 1 2	4 3 3	2 1 0
P ₁	2 1 2	3 2 2	
P ₂	4 0 1	9 0 2	
P ₃	0 2 0	7 5 3	
P ₄	1 1 2	1 1 2	

Solution:

Process	Need		
	A	B	C
P ₀	3	2	1
P ₁	1	1	0
P ₂	5	0	1
P ₃	7	3	3
P ₄	0	0	0

Solution:

2. Now, we check for a safe state

Safe sequence:

1. For process P_0 , Need = (3, 2, 1) and

$$\text{Available} = (2, 1, 0)$$

$$\text{Need} ? \text{Available} = \text{False}$$

So, the system will move for the next process.

2. For Process P_1 , Need = (1, 1, 0)

$$\text{Available} = (2, 1, 0)$$

$$\text{Need} ? \text{Available} = \text{True}$$

Request of P_1 is granted.

$$? \quad \text{Available} = \text{Available} + \text{Allocation}$$

$$= (2, 1, 0) + (2, 1, 2)$$

$$= (4, 2, 2) \text{ (New Available)}$$

3. For Process P_2 , Need = (5, 0, 1)

$$\text{Available} = (4, 2, 2)$$

$$\text{Need} ? \text{Available} = \text{False}$$

So, the system will move to the next process.

4. For Process P_3 , Need = (7, 3, 3)

$$\text{Available} = (4, 2, 2)$$

$$\text{Need} ? \text{Available} = \text{False}$$

So, the system will move to the next process.

5. For Process P_4 , Need = (0, 0, 0)

$$\text{Available} = (4, 2, 2)$$

$$\text{Need} ? \text{Available} = \text{True}$$

Request of P_4 is granted.

$$? \quad \text{Available} = \text{Available} + \text{Allocation}$$

$$= (4, 2, 2) + (1, 1, 2)$$

$$= (5, 3, 4) \text{ now, (New Available)}$$

Solution:

6. Now again check for Process P_2 , Need = (5, 0, 1)

Available = (5, 3, 4)

Need ? Available = True

Request of P_2 is granted.

? Available = Available + Allocation
= (5, 3, 4) + (4, 0, 1)
= (9, 3, 5) now, (New Available)

7. Now again check for Process P_3 , Need = (7, 3, 3)

Available = (9, 3, 5)

Need ? Available = True

Request of P_3 is granted.

? Available = Available + Allocation
= (9, 3, 5) + (0, 2, 0) = (9, 5, 5)

8. Now again check for Process P_0 , = Need (3, 2, 1)

= Available (9, 5, 5)

Need ? Available = True

So, the request will be granted to P_0 .

Safe sequence: < P_1 , P_4 , P_2 , P_3 , P_0 >

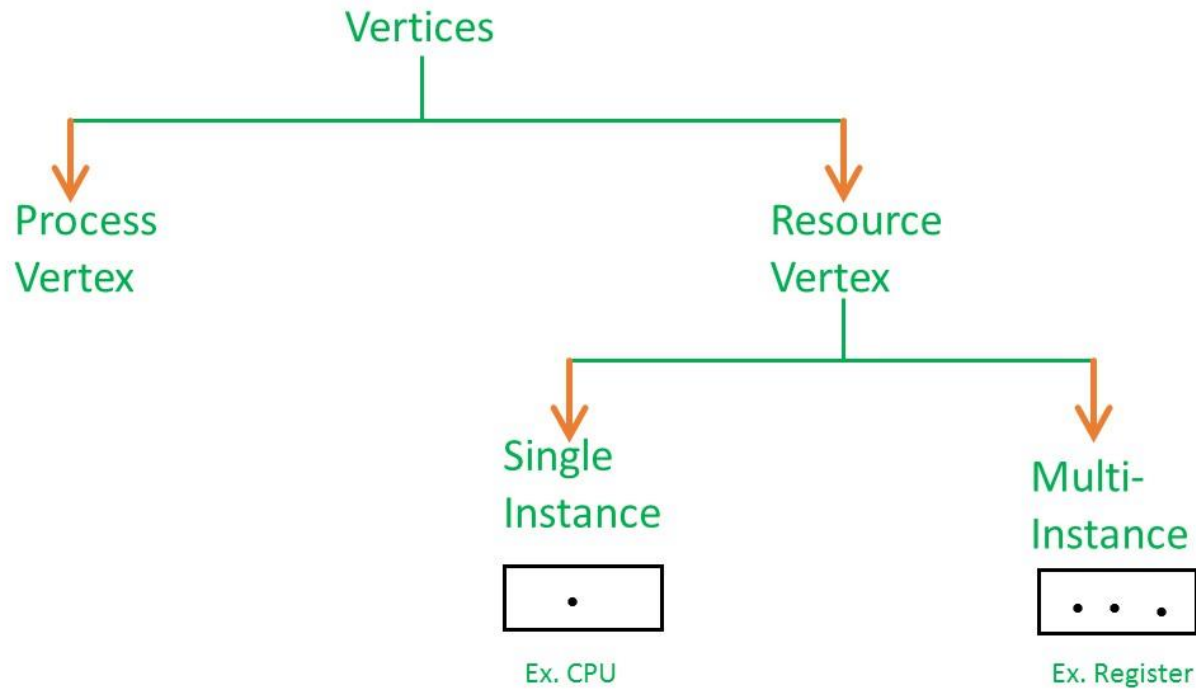
Resource Allocation Graph (RAG) in Operating System:

- As [Banker's algorithm](#) using some kind of table like allocation, request, available all that thing to understand what is the state of the system.
- Similarly, if you want to understand the state of the system instead of using those table, actually tables are very easy to represent and understand it, but then still you could even represent the same information in the graph. That graph is called **Resource Allocation Graph (RAG)**.

Resource Allocation Graph (RAG)

- We know that any graph contains vertices and edges. So RAG also contains vertices and edges. In RAG vertices are two type:
 1. **Process vertex:** Every process will be represented as a process vertex. Generally, the process will be represented with a circle.
 2. **Resource vertex:** Every resource will be represented as a resource vertex. It is also two type:
 - **Single instance type resource:** It represents as a box, inside the box, there will be one dot. So the number of dots indicate how many instances are present of each resource type.
 - **Multi-resource instance type resource:** It also represents as a box, inside the box, there will be many dots present.

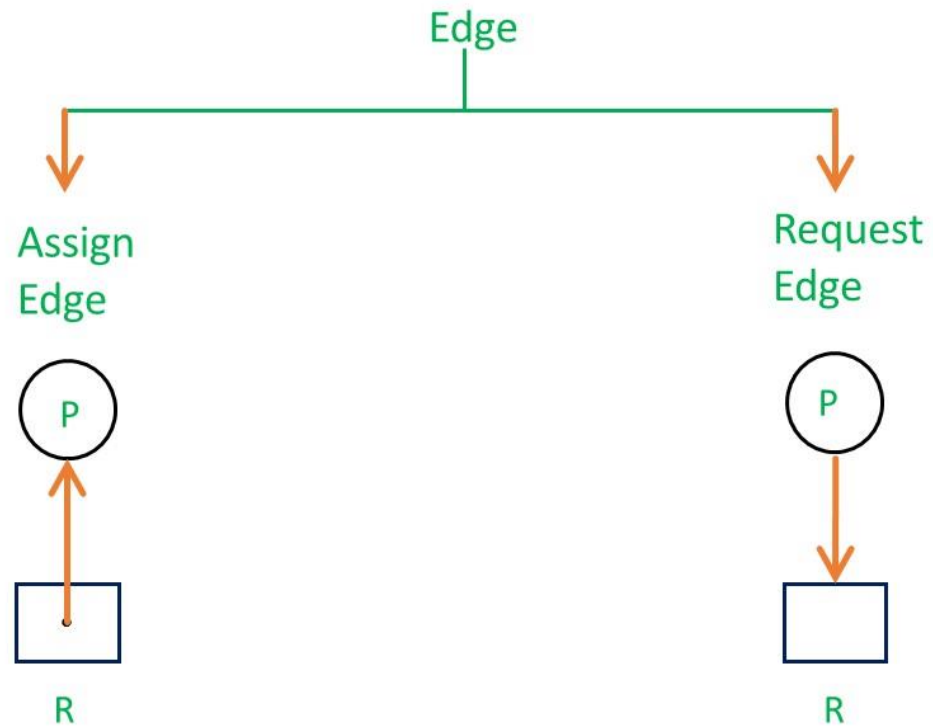
Resource Allocation Graph (RAG)



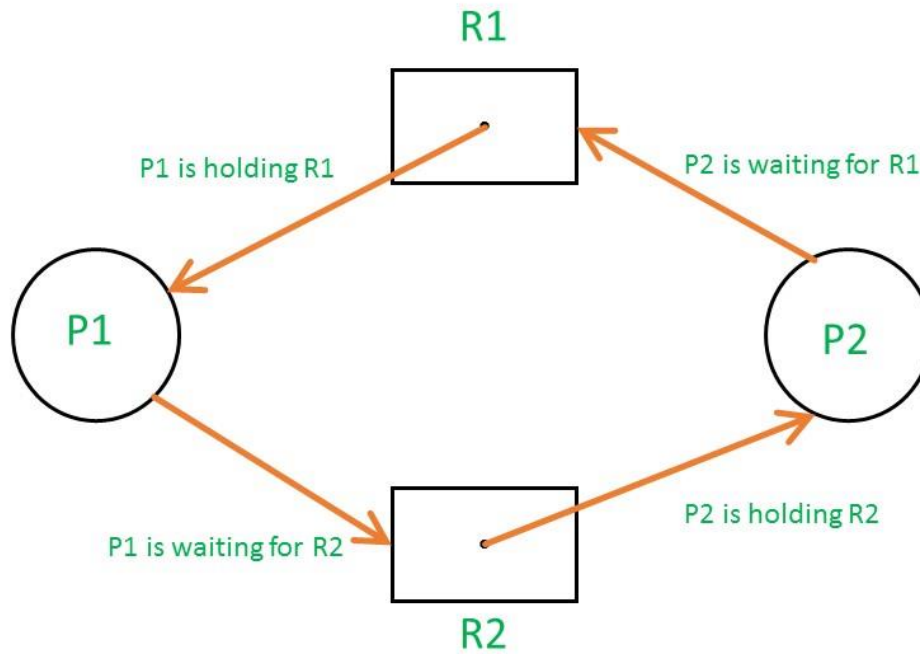
Resource Allocation Graph (RAG)

- Now coming to the edges of RAG. There are two types of edges in RAG:
 1. **Assign Edge:** If you already assign a resource to a process then it is called Assign edge.
 2. **Request Edge:** It means in future the process might want some resource to complete the execution, that is called request edge.
 3. So, if a process is using a resource, an arrow is drawn from the resource node to the process node. If a process is requesting a resource, an arrow is drawn from the process node to the resource node.

Resource Allocation Graph (RAG)

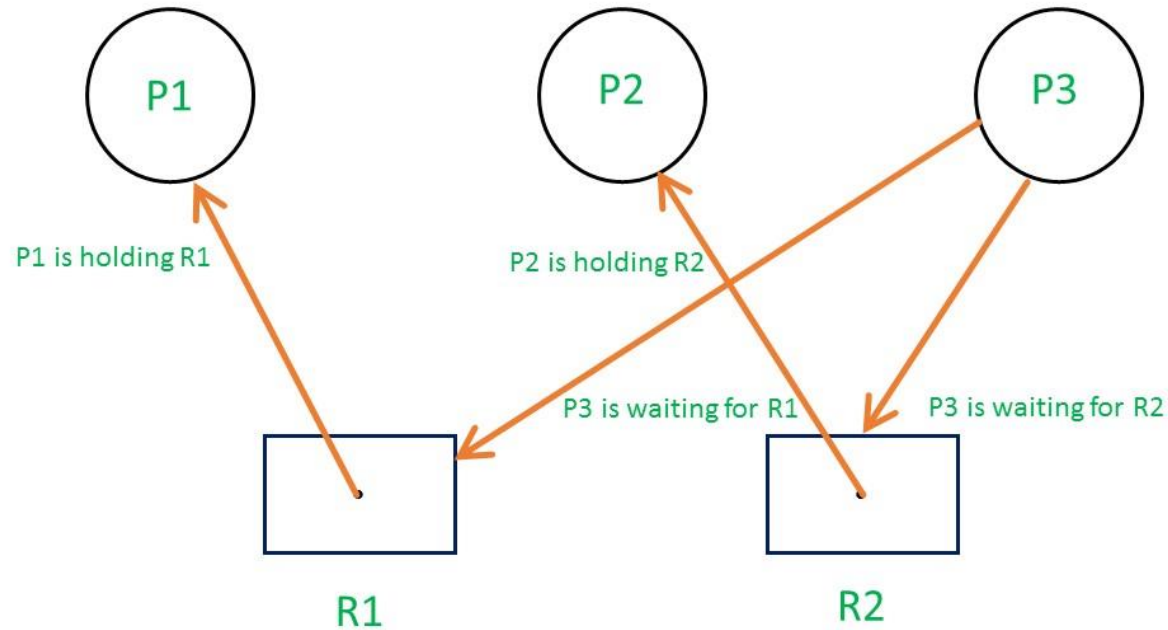


Example 1 (Single instances RAG):



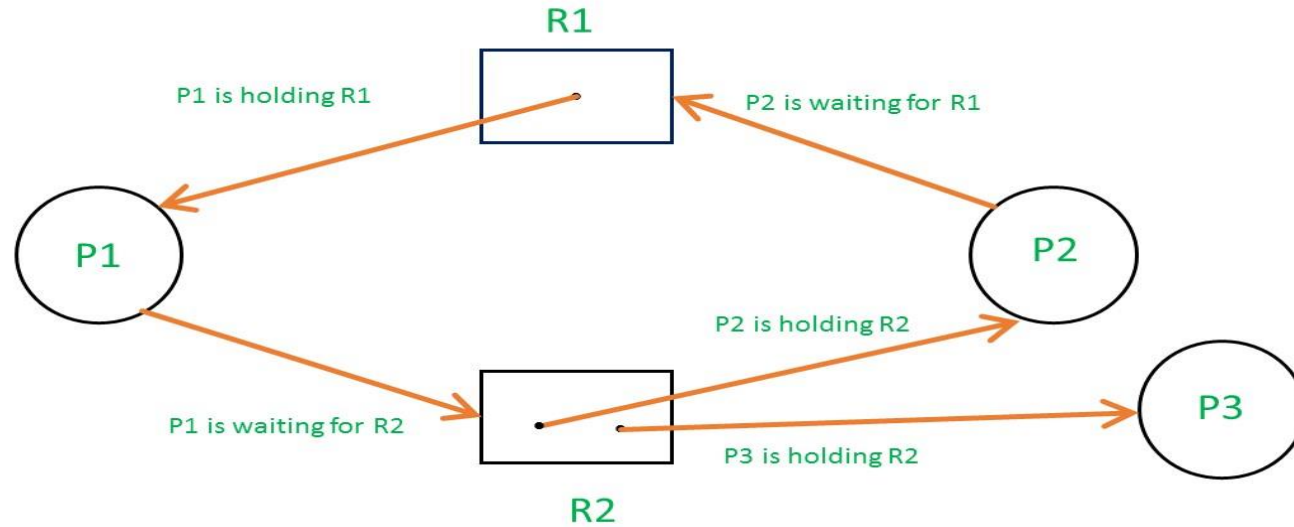
SINGLE INSTANCE RESOURCE TYPE WITH DEADLOCK

Example 1 (Single instances RAG):



SINGLE INSTANCE RESOURCE TYPE WITHOUT DEADLOCK

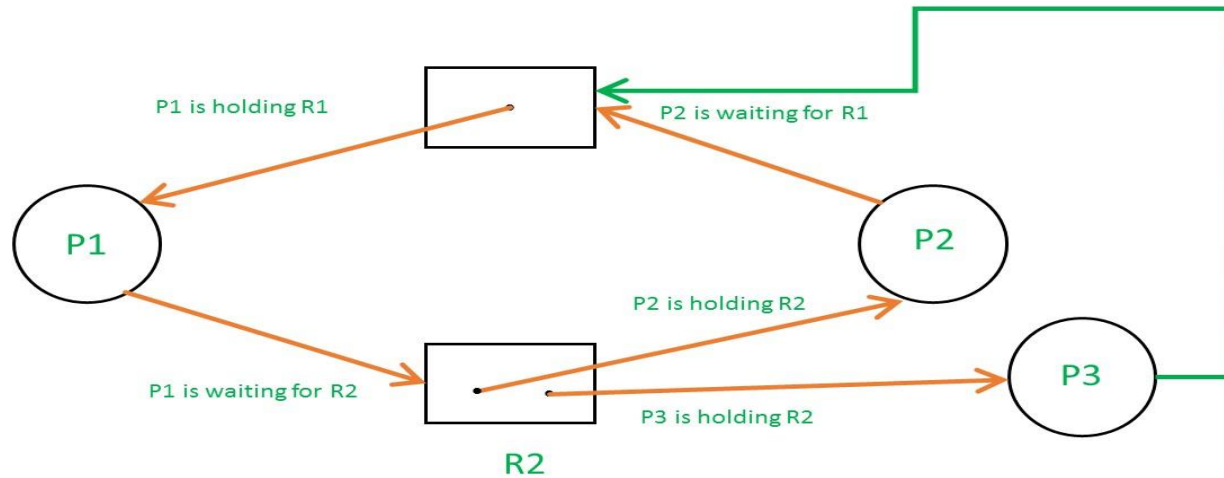
Example 2 (Multi-instances RAG)



MULTI INSTANCES WITHOUT DEADLOCK

Process	Allocation		Request	
	Resource		Resource	
	R1	R2	R1	R2
P1	1	0	0	1
P2	0	1	1	0
P3	0	1	0	0

Example 2 (Multi-instances RAG)



MULTI INSTANCES WITH DEADLOCK

Process	Allocation		Request	
	Resource		Resource	
	R1	R2	R1	R2
P1	1	0	0	1
P2	0	1	1	0
P3	0	1	1	0

EXAMPLES

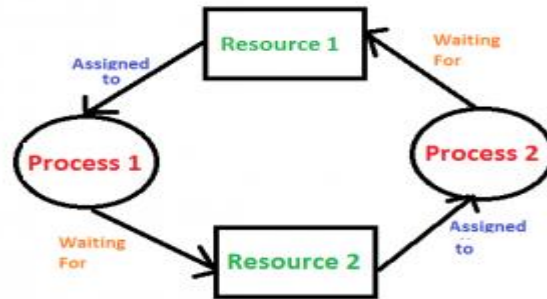
- **EXAMPLE 1:**
- No. of processes= 2, Number of instances of R required by each process = 2, For deadlock free system, minimum R = ??
- **Ans: 3**
- **EXAMPLE 2:**
- No. of processes= 4, Number of instances of R required by each process = 6, For deadlock free system, minimum R = ??
- **Ans: 21**
- **EXAMPLE 3:** A system contains 16 instances of resource R and if each process allowed to request 3 instances. How many process can be processed in deadlock free system.
- **Ans: 7**
- **EXAMPLE 4:** A system contains 11 instances of resource R and if each process allowed to request 4 instances. How many process can be processed in deadlock free system.
- **Ans: 3**

Deadlock Detection And Recovery

- **Deadlock Detection**

1. **If resources have single instance:**

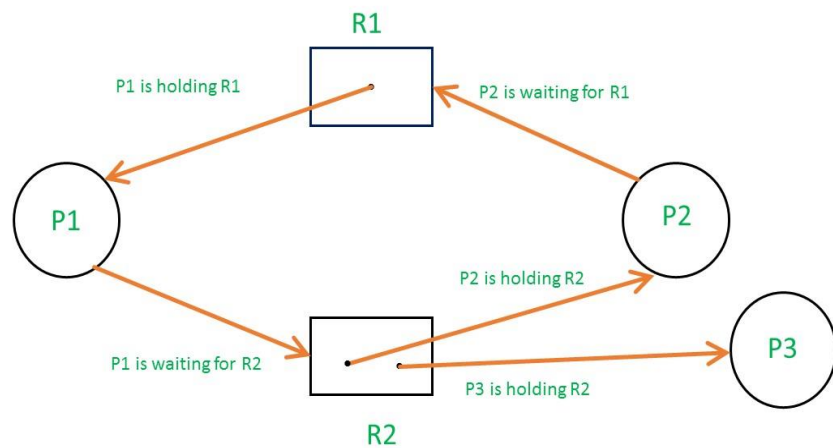
In this case for Deadlock detection we can run an algorithm to check for **cycle** in the *Resource Allocation Graph*. Presence of cycle in the graph is the **sufficient** condition for deadlock.



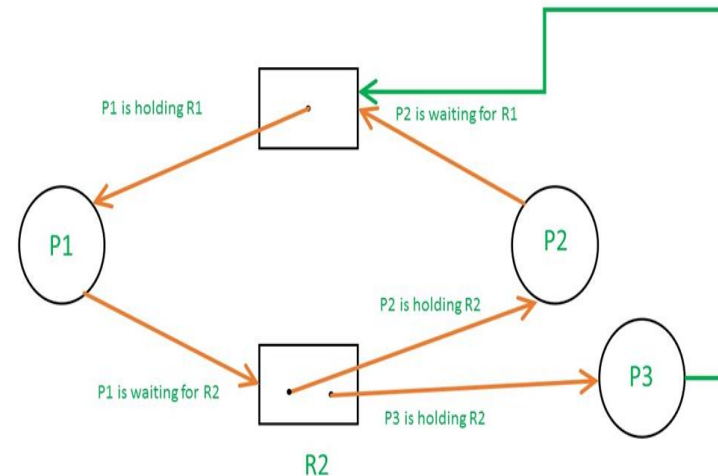
- In the above diagram, resource 1 and resource 2 have single instances. There is a cycle $R1 \rightarrow P1 \rightarrow R2 \rightarrow P2$. So, Deadlock is Confirmed.

Deadlock Detection And Recovery

- If there are multiple instances of resources:
 - Detection of the cycle is necessary but not sufficient condition for deadlock detection, in this case, the system may or may not be in deadlock varies according to different situations.



MULTI INSTANCES WITHOUT DEADLOCK



MULTI INSTANCES WITH DEADLOCK

Deadlock Recovery

- A traditional operating system such as Windows doesn't deal with deadlock recovery as it is time and space consuming process.
- Real-time operating systems use Deadlock recovery.

1.Recovery method Killing the process:

- killing all the process involved in the deadlock.
- Killing process one by one. After killing each process check for deadlock again keep repeating the process till system recover from deadlock.

2. Resource Preemption:

- Resources are preempted from the processes involved in the deadlock, preempted resources are allocated to other processes so that there is a possibility of recovering the system from deadlock.
- In this case, the system goes into **starvation**.